

Практика 10. Монады

Андрей Стоян, Илья Колегов, Дмитрий Халанский

MSE ITMO

осень 2025

В предыдущих сериях

- ▶ Классы типов `Functor` и `Applicative`
- + Класс типов `Monad`
- + Стандартные представители: `Identity`, `Maybe` и `[]`

Содержание

1. Чистые вычисления
2. Работа с монадами
3. Монады в дикой природе (☁️)
4. Материалы

Содержание

1. Чистые вычисления

2. Работа с монадами

3. Монады в дикой природе (☀️)

4. Материалы

Чистые (pure) вычисления

Хорошие свойства функций:

Чистые (pure) вычисления

Хорошие свойства функций:

Идемпотентность

Множественный вызов функции эквивалентен однократному.

Чистые (pure) вычисления

Хорошие свойства функций:

Идемпотентность

Многократный вызов функции эквивалентен однократному.

Ссылочная прозрачность (referential transparency)

Вызов функции можно заменить на её результат без изменения семантики программы.

Чистые (pure) вычисления

Хорошие свойства функций:

Идемпотентность

Многократный вызов функции эквивалентен однократному.

Ссылочная прозрачность (referential transparency)

Вызов функции можно заменить на её результат без изменения семантики программы.

Чистота

- ▶ Единственный наблюдаемый результат функции — итоговое значение (нет побочных эффектов)
- ▶ Функция всегда возвращает один и тот же результат при одном наборе аргументов

Чистые (pure) вычисления

Хорошие свойства функций:

Идемпотентность

Многократный вызов функции эквивалентен однократному.

Ссылочная прозрачность (referential transparency)

Вызов функции можно заменить на её результат без изменения семантики программы.

Чистота

- ▶ Единственный наблюдаемый результат функции — итоговое значение (нет побочных эффектов)
- ▶ Функция всегда возвращает один и тот же результат при одном наборе аргументов

Чистый язык

Позволяет использовать только чистые функции.

Зачем функции быть чистой?

Зачем функции быть чистой?

- ▶ Для простоты рассуждений (reasoning) о программе
 - ▶ Функции всегда ведут себя одинаково
 - ▶ Можно упрощать выражения без потери смысла
 - ▶ Об их поведении можно формулировать алгебраические законы
 - ▶ А зная законы, можно механически и без опасений рефакторить код
 - ▶ Можно проводить более высокоуровневые рассуждения вместо исполнения кода в голове

Зачем функции быть чистой?

- ▶ Для простоты рассуждений (reasoning) о программе
 - ▶ Функции всегда ведут себя одинаково
 - ▶ Можно упрощать выражения без потери смысла
 - ▶ Об их поведении можно формулировать алгебраические законы
 - ▶ А зная законы, можно механически и без опасений рефакторить код
 - ▶ Можно проводить более высокоуровневые рассуждения вместо исполнения кода в голове
- ▶ Для композируемости $f \circ g$
 - ▶ Композиция чистых функций — чистая функция
 - ▶ Сколь угодно сложные и глубокие композиции остаются предсказуемыми
 - ▶ Нет неявных зависимостей, поверхность абстракции наглядно видна

Зачем функции быть чистой?

- ▶ Для простоты рассуждений (reasoning) о программе
 - ▶ Функции всегда ведут себя одинаково
 - ▶ Можно упрощать выражения без потери смысла
 - ▶ Об их поведении можно формулировать алгебраические законы
 - ▶ А зная законы, можно механически и без опасений рефакторить код
 - ▶ Можно проводить более высокоуровневые рассуждения вместо исполнения кода в голове
- ▶ Для композируемости $f \circ g$
 - ▶ Композиция чистых функций — чистая функция
 - ▶ Сколь угодно сложные и глубокие композиции остаются предсказуемыми
 - ▶ Нет неявных зависимостей, поверхность абстракции наглядно видна
- ▶ Для тестируемости

Зачем функции быть чистой?

- ▶ Для простоты рассуждений (reasoning) о программе
 - ▶ Функции всегда ведут себя одинаково
 - ▶ Можно упрощать выражения без потери смысла
 - ▶ Об их поведении можно формулировать алгебраические законы
 - ▶ А зная законы, можно механически и без опасений рефакторить код
 - ▶ Можно проводить более высокоуровневые рассуждения вместо исполнения кода в голове
- ▶ Для композируемости $f \circ g$
 - ▶ Композиция чистых функций — чистая функция
 - ▶ Сколь угодно сложные и глубокие композиции остаются предсказуемыми
 - ▶ Нет неявных зависимостей, поверхность абстракции наглядно видна
- ▶ Для тестируемости
 - ▶ Требуется только варьировать входные параметры и смотреть результат (а не готовить и считывать конфигурацию всего мира вокруг)

Почему не обойтись чистыми функциями?

Почему не обойтись чистыми функциями?

Всё придётся делать самому:

```
parsePolish :: [Token] -> Maybe ([Token], Expr)
parsePolish ts = case parseBinOp ts of
  Nothing -> parseConst ts
  Just (ts', op) -> case parsePolish ts' of
    Nothing -> Nothing
    Just (ts'', left) -> case parsePolish ts'' of
      Nothing -> Nothing
      Just (rest, right) -> Just (rest, BinOp op left right)
```

Почему не обойтись чистыми функциями?

Всё придётся делать самому:

```
parsePolish :: [Token] -> Maybe ([Token], Expr)
parsePolish ts = case parseBinOp ts of
  Nothing -> parseConst ts
  Just (ts', op) -> case parsePolish ts' of
    Nothing -> Nothing
    Just (ts'', left) -> case parsePolish ts'' of
      Nothing -> Nothing
      Just (rest, right) -> Just (rest, BinOp op left right)
```

Для работы со сложностью нужно “заметать под ковёр” неинтересные детали.

Содержание

1. Чистые вычисления

2. Работа с монадами

3. Монады в дикой природе (☀️)

4. Материалы

Монады

Moggi's principle

«Computations of type α correspond to values of type $f \alpha$ »

```
class Applicative m => Monad m where  
  (>>=) :: m a -> (a -> m b) -> m b
```

- Средство последовательной композиции вычислений

todo Реализуйте `liftA2` через `(>>=)`

todo Реализуйте `(>>=)` через `join` :: `m (m a) -> m a`

Монады

Moggi's principle

«Computations of type α correspond to values of type $f \alpha$ »

```
class Applicative m => Monad m where
  (>>=) :: m a -> (a -> m b) -> m b
```

- Средство последовательной композиции вычислений

todo Реализуйте `liftA2` через `(>>=)`

todo Реализуйте `(>>=)` через `join :: m (m a) -> m a`

⚡ `liftA2` оп `mx my = mx >>= \x -> my >>= \y -> pure (x + y)`

Монады

Moggi's principle

«Computations of type α correspond to values of type $f \alpha$ »

```
class Applicative m => Monad m where
  (>>=) :: m a -> (a -> m b) -> m b
```

- Средство последовательной композиции вычислений

todo Реализуйте `liftA2` через `(>>=)`

todo Реализуйте `(>>=)` через `join :: m (m a) -> m a`

⚡ `liftA2` оп `mx my = mx >>= \x -> my >>= \y -> pure (x + y)`

⚡ Bind часто называют `flatMap`

```
m >>= k = join $ k <$> m
```

Монада `Either`

```
class Applicative m => Monad m where  
  (>>=) :: m a -> (a -> m b) -> m b
```

todo Сделайте `Either` монадой

todo Реализуйте `join` для `Either`

todo Найдите имена всех котов человека, если База Котов отвечает

```
data Person { personName :: String, personCats :: [CatId] }  
getPerson :: Db -> PersonId -> Either DbError Person  
getCat :: Db -> CatId -> Either DbError Cat
```

Монада `Either`

```
class Applicative m => Monad m where
  (>=) :: m a -> (a -> m b) -> m b
```

todo Сделайте `Either` монадой

todo Реализуйте `join` для `Either`

todo Найдите имена всех котов человека, если База Котов отвечает

```
data Person { personName :: String, personCats :: [CatId] }
getPerson :: Db -> PersonId -> Either DbError Person
getCat :: Db -> CatId -> Either DbError Cat
```



```
instance Monad (Either e) where
  (>=) :: Either e a -> (a -> Either e b) -> Either e b
  m >= k = case m of Left e -> Left e; Right x -> k x
```

Монада Either

```
class Applicative m => Monad m where
  (>=) :: m a -> (a -> m b) -> m b
```

todo Сделайте `Either` монадой

todo Реализуйте `join` для `Either`

todo Найдите имена всех котов человека, если База Котов отвечает

```
data Person { personName :: String, personCats :: [CatId] }
getPerson :: Db -> PersonId -> Either DbError Person
getCat :: Db -> CatId -> Either DbError Cat
```

```
⚡ instance Monad (Either e) where
  (>=) :: Either e a -> (a -> Either e b) -> Either e b
  m >= k = case m of Left e -> Left e; Right x -> k x
```

```
⚡ catsOf :: Db -> PersonId -> Either DbError [Cat]
catsOf db pid = getPerson db pid >= \person ->
  traverse (getCat db) (personCats person)
```

Монада списка

`todo` Реализуйте монаду списка

`todo` Реализуйте `join` для монады списка

`todo` Реализуйте `filter`, используя монаду списка

`todo` Функция `next :: Board -> [Board]` выдаёт всевозможные состояния шахматной доски после следующего хода, может ли игра закончиться за `n` ходов (`canFinish :: Int -> Board -> Bool`)?

Монада списка

todo Реализуйте монаду списка

todo Реализуйте `join` для монады списка

todo Реализуйте `filter`, используя монаду списка

todo Функция `next :: Board -> [Board]` выдаёт всевозможные состояния шахматной доски после следующего хода, может ли игра закончиться за `n` ходов
(`canFinish :: Int -> Board -> Bool`)?



`(>>=) = flip concatMap`

Монада списка

`todo` Реализуйте монаду списка

`todo` Реализуйте `join` для монады списка

`todo` Реализуйте `filter`, используя монаду списка

`todo` Функция `next :: Board -> [Board]` выдаёт всевозможные состояния шахматной доски после следующего хода, может ли игра закончиться за `n` ходов (`canFinish :: Int -> Board -> Bool`)?

⚡ `(>>=) = flip concatMap`

⚡ `filter p xs = xs >>= \x -> if p x then [x] else []`

Монада списка

`todo` Реализуйте монаду списка

`todo` Реализуйте `join` для монады списка

`todo` Реализуйте `filter`, используя монаду списка

`todo` Функция `next :: Board -> [Board]` выдаёт всевозможные состояния шахматной доски после следующего хода, может ли игра закончиться за `n` ходов (`canFinish :: Int -> Board -> Bool`)?



```
(>>=) = flip concatMap
```



```
filter p xs = xs >>= \x -> if p x then [x] else []
```



```
canFinish :: Int -> Board -> Bool
```

```
canFinish nSteps board = any finished $ allOutcomes nSteps board
```

```
where
```

```
allOutcomes nSteps board
```

```
  | nSteps <= 0 = board
```

```
  | otherwise = next board >>= allOutcomes (nSteps - 1)
```

“Консоль” и `do`-нотация

`todo` Два числа с “консоли”, поделите одно на другое нацело и распечатайте результат, если остаток не нулевой, распечатайте его тоже



“Консоль” и `do`-нотация

`todo` Два числа с “консоли”, поделите одно на другое нацело и распечатайте результат, если остаток не нулевой, распечатайте его тоже



Через явное связывание

```
main :: MyIO ()
main =
  readInt >>= \n ->
  readInt >>= \k ->
  let (d, m) = n `divMod` k in
  writeInt d *>
  if m == 0 then pure () else
    writeInt m
```

“Консоль” и `do`-нотация

`todo` Два числа с “консоли”, поделите одно на другое нацело и распечатайте результат, если остаток не нулевой, распечатайте его тоже



Через явное связывание

```
main :: MyIO ()
main =
  readInt >>= \n ->
  readInt >>= \k ->
  let (d, m) = n `divMod` k in
  writeInt d *>
  if m == 0 then pure () else
    writeInt m
```

Синтаксический сахар

```
main :: MyIO ()
main = do
  n <- readInt
  k <- readInt
  let (d, m) = n `divMod` k
  writeInt d
  if m == 0 then pure () else
    writeInt m
```

Иначе говоря,



Maksim Khlystov 7 лет назад

Мы не в Матрице, мы в Монаде.



38



[Ответить](#)



Содержание

1. Чистые вычисления

2. Работа с монадами

3. Монады в дикой природе (☀️)

4. Материалы

Обработка ошибок

Открываем документацию `java.util.Optional<T>` и видим¹:

► `<U> Optional<U> map(Function<T, U> mapper)`

⇒ Да это же функтор!

```
map :: forall u. Optional t -> (t -> u) -> Optional u
```

► `static <T> Optional<T> of(T value)`

```
<U> Optional<U> flatMap(Function<T, Optional<U>> mapper)
```

⇒ Да это же монада!

```
of :: forall t. t -> Optional t
```

```
flatMap :: forall u. Optional t -> (t -> Optional u) -> Optional u
```

¹Из типов убраны wildcards для простоты.

Ненатуральный пример из дикой природы

Java

```
Optional<Integer> secureSum(Long uid, AuthDb auth, Db1 db1, Db2 db2) {  
    return  
        auth.check(uid).flatMap(_ ->  
            db1.lookup(uid).flatMap(res1 ->  
                db2.lookup(uid).flatMap(res2 ->  
                    Optional.of(res1 + res2)))));  
}
```

Haskell

```
secureSum :: UserId -> AuthDb -> Db1 -> Db2 -> Maybe Int  
secureGet uid auth db1 db2 = do  
    check auth user  
    res1 <- db1 `Db.find` user  
    res2 <- db2 `Db.find` user  
    pure (res1 + res2)
```

Работа с последовательностями

Открываем документацию `java.util.stream.Stream<T>` и видим:

- ▶ `<R> Stream<R> map(Function<T, R> mapper)`
- ▶ `<R> Stream<R> flatMap(Function<T, Stream<R>> mapper)`

Более того, `Stream`'ы являются ленивыми:

- ▶ Применение операции порождает объект, запоминающий новое преобразование
- ▶ Чтобы форсировать вычисление нужно воспользоваться свёрткой `java.util.stream.Collectors`

Асинхронное программирование

- ▶ Оператор “точка с запятой” — дождаться выполнения предыдущей инструкции и перейти к следующей
 - ▶ “Дождаться” — обычно заблокировать нативный поток — мб очень дорого
 - ▶ Нужно чтобы “точка с запятой” запоминала, что сделать по готовности результата вместо блокирования потока исполнения
 - ▶ “Перегружаем” оператор “точка с запятой”!
- ⇒ Заводим объект: коробочка, в которой потом появится значение²
- ⇒ Заводим операцию для него: запомни, что сделать по готовности результата

Блокирующий код

```
var c = fetch("config");  
var res = apiRequest1(c);  
return apiRequest2(res);
```

Асинхронный код

```
return  
  fetch("config").then((c) =>  
    apiRequest1(c)).then((res) =>  
    apiRequest2(res))
```

- ▶ Если запросы независимы, то их можно исполнить параллельно — аппликативы

²Такие объекты называют Future, Promise или Deferred...

Содержание

1. Чистые вычисления

2. Работа с монадами

3. Монады в дикой природе (☀️)

4. Материалы

Что посмотреть в транспорте

- ▶ Vitaly Bragilevsky - The clear path to Haskell complexities
- ▶ Виталий Брагилевский — Монады - не приговор

- ▶ Moggi, Eugenio. *Computational lambda-calculus and monads*. University of Edinburgh, Department of Computer Science, Laboratory for Foundations of Computer Science, 1988.
- ▶ Wadler, Philip. "Comprehending monads." *Proceedings of the 1990 ACM Conference on LISP and Functional Programming*. 1990.
- ▶ Wadler, Philip. "The essence of functional programming." *Proceedings of the 19th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. 1992.